

OOP 期末考试纸质资料（黑白打印版）

选择题 + 代码审阅题速查（例题化增强版）

目录

1 使用说明	4
2 考试总框架与优先级	4
2.1 题型与资料使用思路	4
2.2 最高频知识点排序	4
2.3 选择题通用判断顺序（例题化版）	4
3 代码阅读最小错例库：先判断能否编译	5
3.1 const 成员函数调用非 const 接口	5
3.2 static 函数直接访问 this 或非静态成员	6
3.3 override 是否真的匹配基类 virtual 函数	6
3.4 map 的 operator[] 在 const 函数中被调用	7
3.5 引用绑定是否合法	7
3.6 抽象类是否被实例化	8
3.7 综合扫雷例子	8
4 编译、链接、头文件与 Makefile	9
4.1 编译与链接的区别	9
4.2 声明与定义	9
4.3 头文件原则	9
4.4 防止头文件重复包含	10
5 宏、inline、auto、decltype、nullptr	10
5.1 宏定义	10
5.2 inline	10
5.3 函数重载与缺省参数	10
5.4 auto	11
5.5 decltype	11
5.6 nullptr	11
6 类、对象、封装、访问权限	12
6.1 类的本质	12
6.2 访问权限	12
6.3 this 指针	12
7 构造、析构与对象生命周期	12
7.1 构造函数	12
7.2 默认构造函数	13
7.3 初始化列表	13
7.4 析构函数	14
7.5 构造/析构顺序总表	14

7.6 new/delete	15
8 const、static、friend	15
8.1 const 数据成员与 const 成员函数	15
8.1.1 const 题型最小例子	15
8.2 static 数据成员	16
8.3 static 成员函数	16
8.4 friend 友元	16
9 引用、左值、右值、移动语义	16
9.1 普通引用	16
9.2 左值与右值	17
9.3 引用绑定规则	17
9.4 std::move	17
10 拷贝构造、移动构造、赋值运算符	18
10.1 拷贝构造函数	18
10.2 默认拷贝的问题	18
10.3 赋值运算符	18
10.4 移动构造与移动赋值	19
10.5 RVO 与 -fno-elide-constructors	19
11 类型转换与重载解析	19
11.1 内置类型转换	19
11.2 自定义类型转换函数	19
11.3 重载解析优先级粗略记忆	20
12 运算符重载与 iostream	20
12.1 必须作为成员函数重载的运算符	20
12.2 流运算符	20
12.3 函数对象	20
13 组合与继承	21
13.1 组合 has-a	21
13.2 继承 is-a	21
13.3 继承中什么不会被普通继承	21
13.4 访问控制与继承方式	21
14 虚函数、多态、对象切片	22
14.1 对象切片	22
14.2 virtual 与动态绑定	22
14.3 重写覆盖 vs 重写隐藏	22
14.4 override 与 final	23
15 虚析构、抽象类、dynamic_cast	23
15.1 虚析构造函数	23
15.2 纯虚函数与抽象类	24
15.3 纯虚析构造函数	24
15.4 dynamic_cast vs static_cast	24
16 模板、类模板、成员函数模板	24
16.1 函数模板	24
16.2 类模板	25
16.3 模板与虚函数	25
17 STL 容器与迭代器	25
17.1 pair 与 tuple	25

17.2	vector	26
17.3	迭代器	26
17.4	vector 迭代器失效	26
17.5	string	27
17.6	map	27
17.7	set/map 迭代器	27
17.8	list	28
18	智能指针与资源管理	28
18.1	shared_ptr	28
18.2	shared_ptr 的严重错误：同一裸指针构造两个 shared_ptr	28
18.3	unique_ptr	28
18.4	weak_ptr	29
19	iostream、文件流、stringstream	29
19.1	cout/cin 是对象	29
19.2	endl	29
19.3	文件流	29
19.4	stringstream	29
20	设计模式速查	30
20.1	总原则	30
20.2	模板方法 Template Method	30
20.3	策略 Strategy	30
20.4	迭代器 Iterator	31
20.5	适配器 Adapter	31
20.6	代理 Proxy	31
20.7	装饰器 Decorator	31
20.8	单例 Singleton	32
20.9	工厂方法 Factory Method	32
21	代码审阅题：总检查表	32
21.1	正确性问题优先级最高	32
21.2	效率问题	33
21.3	设计合理性问题	33
22	代码审阅题：本次 AI 代码常见问题模板	33
22.1	getPlayCount 的 const 编译错误	33
22.2	removeByArtist 中 erase 后迭代器失效	34
22.3	generateRecommendations 遍历 vector 时 push_back	34
22.4	transferSong move 后继续使用原指针	35
22.5	importSong 中同一裸指针构造两个 shared_ptr	35
22.6	displaySong(Song song) 对象切片	35
22.7	参数传递与 const 正确性	36
23	选择题历年考点速查	36
23.1	2025 选择题考点	36
23.2	23 年选择题考点	36
24	最后一页：考场 30 秒速查	37

1 使用说明

本资料按“**考试时最快能查到答案**”的思路整理，而不是按课件逐页复述。重点来源包括课程课件、两年选择题、代码审阅题和 `oop.zip` 笔记。本版把抽象检查点改成“**最小错例 -> 为什么错 -> 正确写法 -> 选择题识别**”，便于在纸质资料上直接对照代码。

标记规则：

- **加粗**：高频考点、易错点、代码审阅题常用扣分点。
- “判断模板”：选择题快速判断步骤。
- “代码审阅”：遇到大段 AI 代码时优先检查的错误。
- 本资料为黑白打印设计，尽量不用颜色；表格和粗体用于区分层级。

考试建议：选择题先判断“**会不会编译**”，再判断“**调用哪个函数**”，最后判断“**资源是否正确释放**”。代码审阅题先从正确性拿分，再写效率与设计。

2 考试总框架与优先级

2.1 题型与资料使用思路

题型	常见任务	纸质资料怎么用
选择题	判断说法正误、输出、编译错误	查“易错清单”和“调用顺序”
代码审阅题	找 AI 代码问题	查“代码审阅总表”，按正确性/效率/设计分条写

2.2 最高频知识点排序

1. 构造/析构顺序、成员对象、继承对象、`new/delete`。
2. 拷贝构造、移动构造、赋值运算符、右值引用。
3. 虚函数、多态、对象切片、虚析构、`dynamic_cast`。
4. STL 容器、迭代器失效、`map/set/vector/string`。
5. 智能指针：`shared_ptr`、`unique_ptr`、`weak_ptr`。
6. `const/static/friend/this`/访问控制。
7. 函数重载、缺省参数、`auto`、`decltype`、宏、`inline`。
8. 设计模式：策略、模板方法、适配器、代理、装饰器、单例、工厂。

2.3 选择题通用判断顺序（例题化版）

做代码判断题时，不要一开始就算输出。先按下面顺序扫，**每一步都对应一个高频编译错误或运行时错误**。

顺序	先看什么	最小判断例子	结论
1	抽象类	<code>class A{ virtual void f()=0; }; A a;</code>	错：抽象类不能实例化
2	<code>override</code>	基类 <code>virtual void f(float)</code> ， 派生类 <code>void f(int) override</code>	错：参数不匹配，不是覆盖
3	<code>const</code> 成员函数	<code>int get() const { return mp[key]; }</code>	错： <code>map[]</code> 可能插入元素
4	<code>static</code> 成员函数	<code>static void f(){ cout << x; }</code>	错：不能直接访问非静态成员
5	引用绑定	<code>int& r = 1; 或 void f(int&&); f(a);</code>	错：引用类型和左右值不匹配

顺序	先看什么	最小判断例子	结论
6	生命周期	Base* p = new Derived; delete p; 且 ~Base() 非 virtual	危险：派生类析构可能不调用
7	容器迭代器	for(it...) v.erase(it); ++it	错：erase 后 it 失效

第二步：对象生命周期。

- 构造：基类 -> 成员 -> 自身构造函数体。
- 析构：自身析构函数体 -> 成员 -> 基类。
- 局部对象：出作用域就析构。
- new 出来的对象：只有 delete 才触发析构；指针变量出作用域不会自动 delete 它指向的堆对象。

第三步：调用哪个函数？

- 普通函数：早绑定，编译期决定。
- virtual + 基类指针/引用：晚绑定，运行期按实际对象。
- 对象传值：发生对象切片，多态失效。
- 同名不同参：在派生类中是隐藏，不是自动重载。

第四步：资源是否安全？

- new[] 必须配 delete[]。
- 基类指针 delete 派生类对象时，基类析构函数必须是 virtual。
- 一个裸指针不能交给两个 shared_ptr 分别管理。
- vector 遍历时 push_back/erase 可能导致迭代器失效。

3 代码阅读最小错例库：先判断能否编译

这一节专门解决“知道检查点，但不知道代码里长什么样”的问题。考试中看到类似结构，优先判断编译性。

3.1 const 成员函数调用非 const 接口

错例：

```
class A {
    int x = 0;
public:
    void change() { x++; }
    int get() const {
        change();    // 错：const 函数中调用非 const 成员函数
        return x;
    }
};
```

为什么错：get() const 中的 this 可以理解为 const A* this，而 change() 可能修改对象。

正确写法：

```
class A {
    int x = 0;
public:
    int read() const { return x; }
    int get() const { return read(); }
};
```

选择题识别：看到 函数(...) const { ... }, 检查里面调用的成员函数后面有没有 const。

3.2 static 函数直接访问 this 或非静态成员

错例：

```
class A {
    int x = 0;
public:
    static void f() {
        x = 1;           // 错：static 函数没有 this
        // cout << this; // 也错
    }
};
```

为什么错：static 成员函数属于类，不属于某个对象；调用 A::f() 时没有具体对象。

正确写法：

```
class A {
    int x = 0;
    static int cnt;
public:
    static void f(A& a) {
        a.x = 1;         // 对：通过对象访问
        cnt++;           // 对：访问静态成员
    }
};
int A::cnt = 0;
```

选择题识别：看到 static 函数体，检查是否直接用了普通成员变量、普通成员函数或 this。

3.3 override 是否真的匹配基类 virtual 函数

错例 1：参数不匹配。

```
class Base {
public:
    virtual void f(float x) {}
};
class Derived : public Base {
public:
    void f(int x) override {} // 错：int 和 float 不匹配
};
```

错例 2：const 不匹配。

```
class Base {
public:
    virtual void f() const {}
};
class Derived : public Base {
public:
    void f() override {}      // 错：少了 const
};
```

错例 3：基类不是 virtual。

```
class Base { public: void f() {} };
class Derived : public Base {
public:
    void f() override {}      // 错：基类 f 不是 virtual
};
```

正确写法：

```
class Base { public: virtual void f(int) const {} };
class Derived : public Base {
public:
    void f(int) const override {}
};
```

选择题识别：override 要同时满足：基类 virtual、同名、同参数、const 一致、返回值兼容。

3.4 map 的 operator[] 在 const 函数中被调用

错例：

```
#include <map>
#include <string>
using namespace std;

class A {
    map<string, int> mp;
public:
    int get(const string& s) const {
        return mp[s];      // 错：operator[] 可能插入新元素
    }
};
```

为什么错：mp[s] 不只是查询；当 key 不存在时，它会插入 s -> 0，所以不能在 const 成员函数中调用。

正确写法：

```
int get(const string& s) const {
    auto it = mp.find(s);
    if (it == mp.end()) return 0;
    return it->second;
}
```

选择题识别：看到 xxx(...) const { return mapName[key]; }, 大概率是编译错误。

3.5 引用绑定是否合法

代码	是否正确	原因
int a=1; int& r=a;	对	非 const 左值引用绑定左值
int& r=1;	错	非 const 左值引用不能绑定右值
const int& r=1;	对	const 左值引用可以绑定右值
int a=1; int&& r=a;	错	右值引用不能绑定左值
int&& r=1;	对	右值引用绑定右值
int a=1; f(std::move(a));	通常对	std::move(a) 把 a 转成右值表达式

右值引用变量本身是左值：

```
void f(int& x) { cout << "left"; }
void f(int&& x) {
    cout << "right";
    f(x);      // x 有名字，所以 x 是左值，调用 f(int&)
}
int main() { f(1); } // 输出 rightleft
```

选择题识别：判断引用绑定时先问：传进去的表达式有没有名字？有名字通常就是左值。

3.6 抽象类是否被实例化

错例：

```
class Base {
public:
    virtual void f() = 0;
};
int main() {
    Base b;           // 错：抽象类不能定义对象
    Base* p = new Base; // 错：new 也是实例化
}
```

正确写法：

```
class Derived : public Base {
public:
    void f() override {}
};
int main() {
    Derived d;      // 对
    Base* p = &d;    // 对：基类指针可以指向派生类对象
}
```

易错：派生类没实现所有纯虚函数，也还是抽象类。

```
class Base { public: virtual void f()=0; virtual void g()=0; };
class D : public Base { public: void f() override {} };
D d; // 错：g() 没实现，D 仍是抽象类
```

3.7 综合扫雷例子

```
class Base { public: virtual void f(int)=0; };
class A : public Base {
    map<string,int> mp;
    int x = 0;
public:
    void f(double) override {} // 1 错：override 参数不匹配
    int get(string s) const { return mp[s]; } // 2 错：const 中用 map[]
    static void show() { cout << x; } // 3 错：static 直接用非静态成员
};
int main() {
    Base b; // 4 错：抽象类实例化
    int& r = 1; // 5 错：非 const 左值引用绑定右值
}
```

考场做法：先标出 1-5 这种“编译期硬错误”，再考虑输出和资源问题。

4 编译、链接、头文件与 Makefile

4.1 编译与链接的区别

阶段	做什么	常见错误
预处理	展开 <code>#include</code> 、宏	头文件重复包含、宏副作用
编译	检查语法、类型、生成 <code>.o</code>	未声明、类型不匹配、 <code>const</code> 错误
链接	把多个 <code>.o</code> 合成可执行文件	<code>undefined reference</code> 、 <code>multiple definition</code>

常用命令：

```
g++ main.cpp -o main

g++ -std=c++11 main.cpp -o main

g++ -c a.cpp -o a.o      // 只编译不链接
g++ -c b.cpp -o b.o
g++ a.o b.o -o main      // 链接
```

易错： `g++ -c` 的意思是只编译，不链接。

4.2 声明与定义

```
int add(int a, int b);           // 声明
int add(int a, int b) { return a + b; } // 定义
```

同一个函数可以声明多次，但只能定义一次。

变量也一样：

```
extern int x; // 声明，不分配空间
int x = 0;    // 定义，分配空间
```

4.3 头文件原则

头文件通常放：

- 类声明。
- 函数声明。
- 模板定义。
- `inline` 函数定义。

源文件通常放：

- 普通函数定义。
- 类成员函数定义。
- 静态数据成员定义。

易错：普通全局变量、普通函数如果直接定义在 `.h` 里，被多个 `.cpp include` 后容易出现 **multiple definition**。

4.4 防止头文件重复包含

写法一：header guard。

```
#ifndef AGENT_H
#define AGENT_H
// declarations
#endif
```

写法二：#pragma once。

```
#pragma once
// declarations
```

两者作用：防止同一个头文件被重复包含。

5 宏、inline、auto、decltype、nullptr

5.1 宏定义

宏是预处理阶段的文本替换，没有类型检查。

```
#define PI 3.1415
#define SQR(x) ((x) * (x))
```

易错：宏不是函数。

```
#define SQR(x) ((x) * (x))
int i = 1;
int a = SQR(i++); // i++ 会执行两次
```

更推荐：

```
const double PI = 3.1415;
inline int sqr(int x) { return x * x; }
```

5.2 inline

inline 是给编译器的建议，不是命令。

- 写了 inline，编译器可以不内联。
- 没写 inline，编译器也可能自行内联。
- 宏展开由预处理器强制完成。

选择题陷阱：“被编译器内联的函数必须带 inline 关键字”是错的。

5.3 函数重载与缺省参数

函数重载要求：函数名相同，参数列表不同。

```
void print(int x);
void print(double x);
```

不能只靠返回值区分：

```
int f(int x);  
double f(int x); // 错，无法重载
```

缺省参数：

```
void f(int a, int b = 1, int c = 2);
```

有缺省值的参数必须放在后面。

二义性例子：

```
void fun(int a, int b = 1);  
void fun(int a);  
fun(2); // 二义性，编译错误
```

5.4 auto

```
auto a = 1;      // int  
auto b = 1.0;    // double  
auto* p = &a;    // int*
```

注意：

```
auto x;          // 错，必须初始化  
auto a = 1, b = 2.0; // 错，同一声明中类型不一致
```

auto* p = &x 中 **p** 的类型和 **decltype(&x)** 相同。

5.5 decltype

decltype(expr) 获取表达式类型。

```
int x = 1;  
decltype(x) y = 2; // int
```

模板返回值常见写法：

```
template <class T, class U>  
auto add(T a, U b) -> decltype(a + b) {  
    return a + b;  
}
```

5.6 nullptr

NULL 在 C++ 中通常是 0，可能引发重载错误。

```
void f(int);  
void f(int*);  
f(NULL); // 可能调用 f(int)  
f(nullptr); // 调用 f(int*)
```

C++11 后表示空指针优先用 **nullptr**。

6 类、对象、封装、访问权限

6.1 类的本质

类 = 数据成员 + 成员函数。

```
class Student {  
private:  
    int id;  
public:  
    void setId(int x) { id = x; }  
    int getId() const { return id; }  
};
```

6.2 访问权限

权限	类外访问	派生类访问	常见用途
public	可以	可以	对外接口
protected	不可以	可以	给子类使用
private	不可以	不可以直接访问	隐藏实现

class 默认 private, struct 默认 public。

6.3 this 指针

成员函数内部隐藏一个 this 指针，指向当前对象。

```
class A {  
    int x;  
public:  
    void set(int x) {  
        this->x = x;  
    }  
};
```

static 成员函数没有 this 指针。

7 构造、析构与对象生命周期

7.1 构造函数

特点：

- 名字与类名相同。
- 没有返回值类型。
- 可以重载。
- 创建对象时自动调用。

```
class A {  
public:  
    A() {}  
    A(int x) {}  
};
```

7.2 默认构造函数

默认构造函数：无参数，或所有参数都有默认值。

```
class A {  
public:  
    A() {}  
};  
  
A a;      // 调用默认构造  
A b();    // 易错：这是函数声明，不是对象定义
```

若用户定义了其他构造函数，编译器不会自动合成默认构造函数。

```
class A {  
public:  
    A(int x) {}  
};  
A a; // 错，没有默认构造函数
```

可显式恢复：

```
A() = default;
```

也可禁止：

```
A(char) = delete;
```

7.3 初始化列表

```
class A {  
    const int x;  
    int& r;  
public:  
    A(int a, int& b) : x(a), r(b) {}  
};
```

必须用初始化列表的情况：

- **const** 成员。
- 引用成员。
- 没有默认构造函数的成员对象。
- 基类构造函数需要参数。

易错：初始化顺序按成员声明顺序，不按初始化列表书写顺序。

```
class A {
    int x;
    int y;
public:
    A() : y(1), x(y) {} // 实际先初始化 x, 此时 y 未定
};
```

7.4 析构函数

特点:

- 名字为 ~类名。
- 没有返回值。
- 没有参数。
- 不能重载, 一个类只有一个析构函数。

```
class A {
public:
    ~A() {}
};
```

7.5 构造/析构顺序总表

场景	构造顺序	析构顺序
成员对象	成员对象 -> 自身构造函数体	自身析构函数体 -> 成员对象
继承对象	基类 -> 成员对象 -> 派生类构造函数体	派生类析构函数体 -> 成员对象 -> 基类
局部对象	执行到定义处构造	出作用域析构
全局对象	main 前构造	main 后析构
new 对象	new 时构造	delete 时析构

经典例子:

```
class Member {
public:
    Member() { cout << "M"; }
    ~Member() { cout << "~M"; }
};

class Base {
    Member m;
public:
    Base() { cout << "B"; }
    ~Base() { cout << "~B"; }
};

int main() {
    Base* b = new Base();
    delete b;
}
```

输出:

MB~B~M

若去掉 `delete b;`, 不会自动析构 `new` 出来的对象, 所以不会输出 `~B~M`。

7.6 new/delete

```
int* p = new int(10);
delete p;

int* a = new int[10];
delete[] a;
```

`new` 配 `delete`, `new[]` 配 `delete[]`。不能混用。

8 const, static, friend

8.1 const 数据成员与 const 成员函数

```
class A {
    const int id;
public:
    A(int x) : id(x) {}
    int getId() const { return id; }
};
```

规则:

- `const` 数据成员只能初始化, 不能赋值。
- `const` 成员函数不能修改对象状态。
- `const` 对象只能调用 `const` 成员函数。

```
const A a(1);
a.getId(); // 可以
```

8.1.1 const 题型最小例子

```
class A {
    int x = 0;
public:
    void set(int v) { x = v; }
    int get() const {
        // set(1); // 错: const 函数不能调用非 const 成员函数
        return x;
    }
};
```

判断口诀: `const` 对象只能调用 `const` 成员函数; `const` 成员函数内部不能修改普通成员, 也不能调用非 `const` 成员函数。

8.2 static 数据成员

```
class A {  
public:  
    static int cnt;  
};  
  
int A::cnt = 0;
```

规则：

- static 数据成员属于类，所有对象共享。
- 应在类外定义一次。
- .h 中声明，.cpp 中定义。

易错：如果在头文件中定义 static 数据成员，被多个源文件包含，可能链接失败。

8.3 static 成员函数

```
class A {  
    int x;  
    static int cnt;  
public:  
    static int getCnt() {  
        return cnt;  
    }  
};
```

static 成员函数不能直接访问非静态成员，也没有 this 指针。

8.4 friend 友元

友元函数/友元类可以访问 private/protected 成员。

```
class A {  
    int x;  
    friend void print(A& a);  
};  
  
void print(A& a) {  
    cout << a.x;  
}
```

重点：

- 友元不是成员函数。
- 友元不对称：A 说 B 是朋友，不代表 B 也把 A 当朋友。
- 友元不传递：朋友的朋友不是朋友。
- 友元不继承：朋友的孩子不是朋友。

9 引用、左值、右值、移动语义

9.1 普通引用

```
int a = 1;
int& r = a;
r = 2; // a 也变成 2
```

规则：

- 引用必须初始化。
- 引用一旦绑定不能改绑。
- 函数返回引用不能返回局部变量引用。

9.2 左值与右值

类型	判断	例子
左值	有名字、可取地址	a、*p、返回左值引用的函数调用
右值	临时值、通常不可取地址	1、a+b、返回普通对象的函数调用

9.3 引用绑定规则

引用类型	可绑定左值	可绑定 const 左值	可绑定右值
T&	可以	不可以	不可以
const T&	可以	可以	可以
T&&	不可以	不可以	可以

易错：右值引用变量本身是左值，因为它有名字。

```
void f(int& x) { cout << "left"; }
void f(int&& x) {
    cout << "right";
    f(x); // x 有名字，是左值，调用 f(int&)
}

f(1); // 先 right，再 left
```

9.4 std::move

std::move(x) 不移动任何东西，只是把 x 转成右值引用。

```
vector<int> v;
process(std::move(v));
```

使用后，v 仍然存在，但其内容处于“有效但未指定”状态，不应依赖原值。

选择题陷阱：static_cast<T&&>(x) 和 std::move(x) 本质类似，但 std::move 更清晰，不应说 static_cast 比 std::move 更安全。

10 拷贝构造、移动构造、赋值运算符

10.1 拷贝构造函数

```
class A {  
public:  
    A(const A& other) {}  
};
```

调用场景：

```
A a;  
A b(a);    // 拷贝构造  
A c = a;   // 拷贝构造  
void f(A x);  
f(a);      // 按值传参，拷贝构造  
A g();     // 返回对象，可能拷贝/移动，也可能被 RVO 优化
```

拷贝构造参数通常写 `const A&`。不能写 `A(A other)`，否则无限递归。

10.2 默认拷贝的问题

如果类里有裸指针：

```
class A {  
    int* p;  
public:  
    A() { p = new int[10]; }  
    ~A() { delete[] p; }  
};
```

```
A a;  
A b = a; // 默认拷贝：a.p 和 b.p 指向同一块内存
```

后果：

- 改 b 也可能影响 a。
- 析构时重复 delete。
- 造成未定义行为。

有资源管理时，要写析构、拷贝构造、拷贝赋值，或使用智能指针/容器。

10.3 赋值运算符

```
class A {  
public:  
    A& operator=(const A& rhs) {  
        if (this != &rhs) {  
            // copy  
        }  
        return *this;  
    }  
};
```

区别：

```
A b = a; // 拷贝构造，新对象
b = a;   // 拷贝赋值，旧对象已经存在
```

10.4 移动构造与移动赋值

```
class A {
    int* p;
public:
    A(A&& other) : p(other.p) {
        other.p = nullptr;
    }

    A& operator=(A&& rhs) {
        if (this != &rhs) {
            delete[] p;
            p = rhs.p;
            rhs.p = nullptr;
        }
        return *this;
    }
};
```

移动语义的目的：避免大对象或资源对象的深拷贝。

10.5 RVO 与 -fno-elide-constructors

返回对象时，编译器可能做返回值优化 RVO，直接在目标位置构造对象。

若题目说明：

```
-std=c++11 -fno-elide-constructors
```

就要按“不省略拷贝/移动”的规则数构造函数。

11 类型转换与重载解析

11.1 内置类型转换

```
void f(int x);
f(3.8); // 转为 3，向零截断
```

11.2 自定义类型转换函数

```
class MyInt {
    int val;
public:
    operator int() const { return val; }
};
```

如果没有重载 `operator+`:

```
MyInt a;  
a + 1;
```

可能先把 `a` 转为 `int`, 再做整数加法。

易错: `auto b = a + 1;` 中 `b` 可能是 `int`, 不是 `MyInt`。

11.3 重载解析优先级粗略记忆

1. 精确匹配。
2. 标准类型转换。
3. 自定义类型转换。
4. 无法判断则二义性。

12 运算符重载与 `iostream`

12.1 必须作为成员函数重载的运算符

以下四个必须是成员函数:

```
operator=  
operator[]  
operator()  
operator->
```

不能把它们写成友元函数来重载。

12.2 流运算符

```
ostream& operator<<(ostream& out, const A& a) {  
    out << a.value;  
    return out;  
}
```

为什么返回 `ostream&`?

- 避免复制流对象。
- 支持链式调用: `cout << a << b << endl;`

`endl` 不只是换行, 还会 **flush** 缓冲区。

12.3 函数对象

重载 `operator()` 的对象像函数一样使用。

```
class Greater {  
public:  
    bool operator()(int a, int b) const {  
        return a > b;  
    }
```

```

    }
};

sort(v.begin(), v.end(), Greater());

```

`greater<int>()` 是对象，不是函数名。

13 组合与继承

13.1 组合 has-a

一个类包含另一个类对象：

```

class Car {
    Engine e;
    Wheel w;
};

```

构造顺序：成员按声明顺序构造。析构顺序：成员按声明相反顺序析构。

13.2 继承 is-a

```

class Derived : public Base {};

```

`public` 继承表达“Derived 是一种 Base”。

派生类对象构造：

Base 构造 -> 成员对象构造 -> Derived 构造函数体

析构：

Derived 析构函数体 -> 成员对象析构 -> Base 析构

13.3 继承中什么不会被普通继承

- 构造函数通常不继承。
- 析构函数不继承。
- 友元关系不继承。
- 基类赋值运算符本身不作为派生类同签名赋值运算符继承，但派生类隐式赋值会调用基类赋值。

13.4 访问控制与继承方式

基类成员	public 继承后	protected 继承后	private 继承后
public	public	protected	private
protected	protected	protected	private
private	派生类不可直接访问	派生类不可直接访问	派生类不可直接访问

`private` 继承不允许外部做正常向上类型转换。

14 虚函数、多态、对象切片

14.1 对象切片

```
class Base { public: int x; };  
class Derived : public Base { public: int y; };
```

```
Derived d;  
Base b = d; // 对象切片, y 丢失
```

函数传值也会切片:

```
void f(Base b);  
f(d); // 切片
```

多态必须通过基类指针或引用实现。

14.2 virtual 与动态绑定

```
class Base {  
public:  
    virtual void f() { cout << "Base"; }  
};  
class Derived : public Base {  
public:  
    void f() override { cout << "Derived"; }  
};  
  
Derived d;  
Base& r = d;  
r.f(); // Derived
```

条件:

- 基类函数是 virtual。
- 通过基类指针或引用调用。
- 派生类函数参数列表匹配。

14.3 重写覆盖 vs 重写隐藏

重写覆盖:

```
class Base {  
public:  
    virtual void f(float x) {}  
};  
class Derived : public Base {  
public:  
    void f(float x) override {}  
};
```

重写隐藏:

```
class Base {  
public:  
    void f(float x) {}  
};  
class Derived : public Base {  
public:  
    void f(int x) {}  
};
```

此时：

```
Derived d;  
d.f(3.14f); // 调用 Derived::f(int)，输出时可能是 3
```

同名不同参在派生类中常常是隐藏，不是你想象中的重载。

若要让基类函数也参与：

```
class Derived : public Base {  
public:  
    using Base::f;  
    void f(int x) {}  
};
```

14.4 override 与 final

override：要求确实重写了基类虚函数，否则编译错误。

```
void f(int) override; // 如果基类没有同签名 virtual f(int)，报错
```

final：禁止继续重写或继承。

```
class A final {};  
virtual void f() final;
```

15 虚析构、抽象类、dynamic_cast

15.1 虚析构函数

如果类要被当作多态基类使用，析构函数应为 virtual。

```
class Base {  
public:  
    virtual ~Base() = default;  
};
```

经典错误：

```
Base* p = new Derived();  
delete p;
```

若 `Base::~~Base()` 不是 virtual，可能只调用 Base 析构，Derived 资源泄漏。

代码审阅题中看到“基类指针 delete 派生类对象”，立刻检查虚析构。

15.2 纯虚函数与抽象类

```
class Shape {  
public:  
    virtual double area() const = 0;  
};
```

含至少一个纯虚函数的类是抽象类，不能定义对象。

```
Shape s; // 错  
Shape* p; // 可以
```

派生类必须实现全部普通纯虚函数，否则仍是抽象类。

15.3 纯虚析构函数

```
class Base {  
public:  
    virtual ~Base() = 0;  
};  
Base::~~Base() {}
```

纯虚析构函数也必须有函数体。

15.4 dynamic_cast vs static_cast

```
Derived* d = dynamic_cast<Derived*>(basePtr);
```

- 指针转换失败：返回 nullptr。
- 引用转换失败：抛出 `std::bad_cast`。
- 基类通常必须是多态类型，即有 `virtual` 函数。

`static_cast` 向下转型不做运行时检查，可能非法访问。

原则：向上转换通常安全；向下转换优先用 `dynamic_cast`。

16 模板、类模板、成员函数模板

16.1 函数模板

```
template <typename T>  
T sum(T a, T b) {  
    return a + b;  
}
```

16.2 类模板

```
template <typename T>
class Box {
    T data;
public:
    T get() const { return data; }
};

Box<int> b;
```

16.3 模板与虚函数

类模板可以有虚函数：

```
template <typename T>
class Base {
public:
    virtual void f() {}
};
```

普通类可以继承类模板实例：

```
class D : public Base<int> {
public:
    void f() override {}
};
```

成员函数模板不能是 **virtual**。

错误：

```
class A {
public:
    template <typename T>
    virtual void f(T x) {}
};
```

原因：虚函数表需要固定函数入口，成员函数模板可能实例化出不确定数量的函数。

17 STL 容器与迭代器

17.1 pair 与 tuple

```
pair<int, string> p(1, "a");
p.first;
p.second;

auto q = make_pair("abc", 7.8);
```

pair 比较：先比 **first**，再比 **second**。

```
make_pair(1, 4) < make_pair(2, 3); // true
make_pair(1, 4) > make_pair(1, 2); // true
```

tuple 长度编译期确定，不能运行时改变。

```
auto t = make_tuple(1, 2.0, "x");
get<0>(t);
```

`get<i>(t)` 的 `i` 必须是编译期常量。

17.2 vector

vector 是自动扩容数组。

```
vector<int> v;
v.push_back(1);
v.pop_back();
v.size();
v.capacity();
v.clear();
v[i];
```

特点：

- 下标访问快。
- 尾部插入均摊 $O(1)$ 。
- 中间插入/删除慢。
- 扩容时搬迁元素，导致迭代器失效。

17.3 迭代器

```
for (auto it = v.begin(); it != v.end(); ++it) {
    cout << *it;
}
```

`begin()` 指向第一个元素，`end()` 指向最后一个元素之后的位置。

17.4 vector 迭代器失效

操作	失效情况
push_back	若扩容，所有迭代器失效
insert	修改位置及之后迭代器失效；若扩容则全部失效
erase	被删位置及之后迭代器失效
clear	所有迭代器失效

错误写法：

```
for (auto it = v.begin(); it != v.end(); ++it) {
    if (*it == 1) v.erase(it); // it 失效，++it 继续用错
}
```

正确写法：

```
for (auto it = v.begin(); it != v.end(); ) {  
    if (*it == 1) it = v.erase(it);  
    else ++it;  
}
```

遍历时 push_back 也危险:

```
for (auto& x : v) {  
    v.push_back(x); // 可能扩容, 引用/迭代器失效  
}
```

17.5 string

```
string s = "abc";  
s[0];  
s.begin();  
s.end();  
s.c_str();
```

s.end() 指向尾后, 不是最后一个字符。

s.c_str() 返回的指针可能在 string 修改后失效。

17.6 map

```
map<string, int> mp;  
mp["a"]++;
```

operator[] 若 key 不存在, 会插入默认值。

const 成员函数中不能用 **mp[key]**, 因为 **operator[]** 可能修改 map。

正确只读写法:

```
int get(const string& key) const {  
    auto it = mp.find(key);  
    if (it == mp.end()) return 0;  
    return it->second;  
}
```

17.7 set/map 迭代器

set/map 迭代器一般是双向迭代器:

```
++it;  
--it;
```

不能:

```
it + 1;  
it += 2;
```

17.8 list

list 是链表。插入元素通常不会导致其他元素迭代器失效。

选择题陷阱：说 list 插入会使后面所有迭代器失效，通常是错的。

18 智能指针与资源管理

18.1 shared_ptr

```
auto p1 = make_shared<A>();  
auto p2 = p1; // 引用计数 +1
```

对象在最后一个 shared_ptr 销毁时释放。

```
vector<shared_ptr<A>> v;  
auto p = make_shared<A>();  
v.push_back(p); // 引用计数为 2  
v.clear();      // 只清掉 vector 里的那一份，p 还持有对象
```

若外面还有 shared_ptr, clear 不会销毁对象。

18.2 shared_ptr 的严重错误：同一裸指针构造两个 shared_ptr

错误：

```
Song* raw = new Song();  
shared_ptr<Song> sp1(raw);  
shared_ptr<Song> sp2(raw); // 错！两个控制块，重复 delete
```

正确：

```
auto sp1 = make_shared<Song>();  
auto sp2 = sp1;
```

18.3 unique_ptr

```
auto p = make_unique<A>();
```

不能拷贝，只能移动。

```
unique_ptr<A> p2 = p;          // 错  
unique_ptr<A> p2 = std::move(p); // 对，p 变空
```

放入 vector：

```
vector<unique_ptr<A>> v;  
v.push_back(std::move(p));
```

18.4 weak_ptr

weak_ptr 不增加引用计数，用来避免 shared_ptr 循环引用。

```
weak_ptr<A> wp = sp;
if (auto p = wp.lock()) {
    // 对象还存在
}
```

19 iostream、文件流、stringstream

19.1 cout/cin 是对象

- cout 是 ostream 对象。
- cin 是 istream 对象。
- 不是类。

19.2 endl

```
cout << endl;
```

等价于输出换行并刷新缓冲区。

endl 不只是 `\n`。

19.3 文件流

```
ifstream ifs("input.txt");
ofstream ofs("output.txt");
fstream fs("data.txt");
```

- ifstream: 输入文件流，只读。
- ofstream: 输出文件流，只写。
- fstream: 输入输出。

选择题陷阱: **ifstream** 不是 **iostream** 的子类，它主要继承输入流能力，不能同时读写。

19.4 stringstream

```
stringstream ss;
ss << "100 200";
int a, b;
ss >> a >> b;
```

清空:

```
ss.str(""); // 清空缓冲区
ss.clear(); // 清空错误状态位
```

clear() 不是清空内容，而是清状态。

20 设计模式速查

20.1 总原则

设计模式三类：

类别	关注点	本课常见模式
行为型	行为/算法可扩展	模板方法、策略、迭代器
结构型	类/对象结构解耦	适配器、代理、装饰器
创建型	对象创建过程解耦	单例、工厂方法、抽象工厂

核心原则：

- 对接口编程，而不是对实现编程。
- 优先使用组合，而不是继承。
- 开闭原则：对扩展开放，对修改关闭。

选择题陷阱：开闭原则不是“对继承开放，对实现关闭”。

20.2 模板方法 Template Method

定义：在基类中定义算法骨架，把某些步骤延迟到子类实现。

```
class Base {
public:
    void run() {
        step1();
        step2();
    }
protected:
    virtual void step1() = 0;
    virtual void step2() = 0;
};
```

关键词：算法骨架固定，细节由子类实现。

20.3 策略 Strategy

定义：封装一系列算法，使它们可以互相替换。

```
class Strategy {
public:
    virtual void run() = 0;
};

class Context {
    Strategy* s;
public:
    void execute() { s->run(); }
};
```

关键词：组合策略对象，运行时可替换行为。

适合：很多类只是行为不同。

20.4 迭代器 Iterator

提供统一方式访问聚合对象元素，而不暴露内部结构。

STL 迭代器就是典型体现：

```
for (auto it = c.begin(); it != c.end(); ++it) {}
```

20.5 适配器 Adapter

定义：把一个类的接口转换成客户希望的另一个接口。

例子：用 vector 实现 Stack。

```
class Vector2Stack : public Stack {  
    vector<int> data;  
public:  
    void push(int x) { data.push_back(x); }  
    void pop() { data.pop_back(); }  
    int top() { return data.back(); }  
};
```

关键词：接口转换，复用旧类。

20.6 代理 Proxy

代理对象控制对真实对象的访问。

用途：

- 权限控制。
- 延迟加载。
- 远程访问。
- 日志记录。

关键词：控制访问。

20.7 装饰器 Decorator

通过组合动态添加功能，不修改原类。

关键词：增强功能，不改变原类接口。

装饰器 vs 代理：

模式	重点
装饰器	增加功能
代理	控制访问

20.8 单例 Singleton

保证一个类只有一个实例，并提供全局访问点。

常见写法：

```
class Counter {
    Counter() = default;
    Counter(const Counter&) = delete;
    Counter& operator=(const Counter&) = delete;
public:
    static Counter& instance() {
        static Counter c;
        return c;
    }
};
```

注意：

- 构造函数私有/受保护。
- 删除拷贝构造和赋值。
- 若担心 delete &instance()，析构函数也要控制访问。
- static 函数不能 virtual。

20.9 工厂方法 Factory Method

封装创建对象逻辑。

```
class TeaFactory {
public:
    static Tea* create(string type) {
        if (type == "green") return new GreenTea;
        if (type == "black") return new BlackTea;
        return nullptr;
    }
};
```

用途：

- 构造逻辑复杂。
- 根据类型创建不同派生类对象。
- 隐藏具体类名。

21 代码审阅题：总检查表

代码审阅题建议按三大类写，每条都尽量写“位置 + 问题 + 后果 + 修改建议”。

21.1 正确性问题优先级最高

检查点	典型错误	后果	修改
编译错误	const 函数调用非 const operator[]	无法编译	用 find/at

检查点	典型错误	后果	修改
迭代器失效	遍历 vector 时 erase 后继续 +it	未定义行为	it = erase(it)
扩容失效	遍历 vector 时 push_back	引用/迭代器失效	先收集，后插入
空指针	move 后继续解引用	崩溃	move 后不使用原指针
重复释放	一个 raw 指针给两个 shared_ptr	double free	用 make_shared 或复制 shared_ptr
对象切片	void f(Base b) 传 Derived	派生信息丢失	const Base&
非虚析构	基类指针 delete 派生对象	派生析构不调用	基类析构 virtual
内存泄漏	new 后没有 delete	泄漏	RAII/智能指针

21.2 效率问题

检查点	典型错误	修改
不必要拷贝	函数参数按值传大对象	改 const T&
shared_ptr 传参	不需要共享所有权却传值	改 const shared_ptr<T>& 或 T&/T*
字符串拷贝	构造函数按值后不 move	使用 std::move 或 const string&
容器查找	每次线性查找	用 map/unordered_map
重复构造	循环中反复创建临时对象	预分配/复用
vector 扩容	已知大小但不 reserve	v.reserve(n)

21.3 设计合理性问题

检查点	错误表现	修改建议
const 正确性	只读函数没加 const	给成员函数加 const
接口权限过大	函数获得修改权限	参数改 const 引用
所有权不清晰	raw pointer 和 shared_ptr 混用	明确所有权，优先智能指针
基类接口	多态基类析构非 virtual	virtual 析构
违反封装	外部直接改内部数据	private + getter/setter
单一职责	一个类做太多事	拆类/策略/工厂

22 代码审阅题：本次 AI 代码常见问题模板

以下模板针对音乐库代码一类题目非常实用。

22.1 getPlayCount 的 const 编译错误

错误模式：

```
int getPlayCount(const string& title) const {
    return playCount[title];
}
```

问题：map::operator[] 在 key 不存在时会插入元素，因此它不是 const 操作。在 const 成员函数中调用会编译失败。

修改：

```
int getPlayCount(const string& title) const {  
    auto it = playCount.find(title);  
    if (it == playCount.end()) return 0;  
    return it->second;  
}
```

22.2 removeByArtist 中 erase 后迭代器失效

错误模式：

```
for (auto it = songs.begin(); it != songs.end(); ++it) {  
    if ((*it)->getArtist() == artist) {  
        songs.erase(it);  
    }  
}
```

问题：erase(it) 后，it 失效，循环末尾 ++it 使用失效迭代器。

修改：

```
for (auto it = songs.begin(); it != songs.end(); ) {  
    if ((*it)->getArtist() == artist) it = songs.erase(it);  
    else ++it;  
}
```

22.3 generateRecommendations 遍历 vector 时 push_back

错误模式：

```
for (auto& song : songs) {  
    if (...) {  
        songs.push_back(make_shared<Song>(*song));  
    }  
}
```

问题：遍历 vector 时 push_back 可能扩容，导致引用 song 和迭代器失效。

修改：先把新增歌曲放到临时容器，遍历结束再插入。

```
vector<shared_ptr<Song>> add;  
for (const auto& song : songs) {  
    if (...) add.push_back(...);  
}  
songs.insert(songs.end(), add.begin(), add.end());
```

另一个问题：make_shared<Song>(*song) 会把 PremiumSong 切成 Song，发生对象切片。若要保留动态类型，应提供虚克隆函数：

```
virtual shared_ptr<Song> clone() const {  
    return make_shared<Song>(*this);  
}
```

派生类重写：

```
shared_ptr<Song> clone() const override {  
    return make_shared<PremiumSong>(*this);  
}
```

22.4 transferSong move 后继续使用原指针

错误模式：

```
void transferSong(shared_ptr<Song> song, Playlist& target) {  
    target.addSong(std::move(song));  
    cout << song->display() << endl;  
}
```

问题：song 被 move 后可能为空，再 song->display() 是空指针解引用。

修改：先输出，再 move；或保存信息。

```
cout << "Transferred: " << song->display() << endl;  
target.addSong(std::move(song));
```

22.5 importSong 中同一裸指针构造两个 shared_ptr

错误模式：

```
Song* raw = new PremiumSong(...);  
shared_ptr<Song> sp1(raw);  
shared_ptr<Song> sp2(raw);
```

问题：sp1、sp2 拥有不同控制块，都会 delete raw，导致重复释放。

正确：

```
auto sp1 = make_shared<PremiumSong>(...);  
auto sp2 = sp1;
```

22.6 displaySong(Song song) 对象切片

错误模式：

```
void displaySong(Song song) {  
    cout << song.display() << endl;  
}  
  
auto s1 = make_shared<PremiumSong>(...);  
displaySong(*s1);
```

问题：按值传入基类对象，派生类部分被切掉，虚函数动态绑定失效。

修改：

```
void displaySong(const Song& song) {  
    cout << song.display() << endl;  
}
```

22.7 参数传递与 const 正确性

可优化:

```
void addSong(shared_ptr<Song> song)
```

若确实要共享所有权, 传值可以接受; 若只是读取或临时使用, 优先:

```
void addSong(const shared_ptr<Song>& song)
```

或更明确:

```
void addSong(shared_ptr<Song> song) { songs.push_back(std::move(song)); }
```

构造函数:

```
Song(string t, string a, int d)
: title(t), artist(a), duration(d) {}
```

会复制字符串。可改:

```
Song(string t, string a, int d)
: title(std::move(t)), artist(std::move(a)), duration(d) {}
```

23 选择题历年考点速查

23.1 2025 选择题考点

题号	高频考点	快速结论
1	宏、auto、decltype、inline	inline 不一定要写关键字, 宏强制展开
2	基类指针 delete 派生对象	基类析构非 virtual 会导致派生析构不调用
3	运算符重载	= [] () -> 必须成员函数
4	右值引用与 RVO	左值不能传给 T&&, 用 std::move 表意更好
5	模板与虚函数	成员函数模板不能是 virtual
6	shared_ptr 引用计数	vector clear 不一定销毁外部仍持有的对象
7	STL 容器	list 插入通常不使后续迭代器全失效
8	设计模式	装饰器通过组合动态添加功能

23.2 23 年选择题考点

题号	高频考点	快速结论
1	static/const/new-delete	static 成员函数没有 this
2	自定义转换、auto、拷贝移动	auto b = a + 1 可能是 int
3	重写覆盖/隐藏	同名不同参是隐藏; f1(float) 是覆盖
4	局部对象析构	test 函数里的对象在 test 结束就析构
5	设计模式	开闭原则: 对扩展开放, 对修改关闭
6	string/vector	end() 指向尾后, 不是最后元素
7	右值引用	具名右值引用变量是左值
8	构造析构顺序	自身析构体先于成员析构

24 最后一页：考场 30 秒速查

先扫编译错误最小特征：`const` 函数里 `map[key]`、`static` 函数直接用普通成员、`override` 参数不匹配、`int& = 1`、抽象类 `A a`、`erase` 后继续用旧迭代器。

1. 构造：基类 -> 成员 -> 自己；析构：自己 -> 成员 -> 基类。
2. `new` 出来的对象不 `delete` 不会自动析构。
3. 析构函数不能重载；构造函数可以重载。
4. `const` 对象只能调用 `const` 成员函数。
5. `static` 成员函数没有 `this`，不能直接访问非静态成员。
6. `const` 成员、引用成员必须初始化列表初始化。
7. 拷贝构造：`A(const A&)`；赋值：`operator=`。
8. 右值引用变量本身是左值。
9. `std::move` 只是类型转换，不真的移动。
10. 对象传值会切片；多态要用指针或引用。
11. 基类指针 `delete` 派生对象，基类析构要 `virtual`。
12. 同名不同参在派生类中常常是隐藏，不是覆盖。
13. `override` 不匹配会编译错误，这是好事。
14. 纯虚函数 `=0`；抽象类不能定义对象。
15. `dynamic_cast` 指针失败返回 `nullptr`，引用失败抛 `bad_cast`。
16. `operator= [] ()` -> 必须是成员函数。
17. `vector erase` 后迭代器失效，要接收返回值。
18. `vector push_back` 可能扩容，使所有迭代器失效。
19. `string/vector` 的 `end` 指向尾后。
20. `map` 的 `operator[]` 不存在会插入，`const` 函数中不能用。
21. 一个裸指针不能构造两个 `shared_ptr`。
22. `unique_ptr` 不能拷贝，只能 `move`。
23. `endl` 会换行并 `flush`。
24. `ifstream` 只读，`ofstream` 只写，`fstream` 读写。
25. 成员函数模板不能是 `virtual`。
26. 策略模式：算法可替换；模板方法：骨架固定。
27. 适配器：接口转换；装饰器：增强功能；代理：控制访问。
28. 单例：构造私有，删除拷贝，返回引用。
29. 开闭原则：对扩展开放，对修改关闭。
30. 代码审阅先写正确性，再写效率，最后写设计。